RAPPORT D'AUDIT DE PERFORMANCE

BarMash

Moteur de simulation temps réel en Go — 10 000 agents, 30 ticks/s

PÉRIMÈTRE DE L'AUDIT

BarMash simule une soirée dans un bar : 10 000 clients se déplacent vers 8 comptoirs, boivent, se bousculent et laissent des flaques qui ralentissent leurs voisins. Le serveur calcule un tick toutes les 33,3 ms et pousse l'état complet au navigateur par WebSocket.

L'audit part de la version naïve livrée, mesure ses goulots réels, puis applique et vérifie une série d'optimisations. Chaque étape est validée par une empreinte bit à bit de l'état de la simulation : le comportement fonctionnel n'a pas changé d'un octet.

Réalisé par **Paul-Antoine CARREAU** • Master 2 Expert en Développement Logiciel (M2 Dev B)

18 octobre 2026

1. Banc d'essai et protocole de mesure

Toutes les mesures ont été faites sur la même machine fixe, **branchée sur secteur, redémarrée juste avant la campagne, sans aucun autre programme démarré à côté**. La baseline et la version finale sont mesurées dans la même session, l'une après l'autre.

Matériel et logiciel

COMPOSANT	SPÉCIFICATION	IMPACT SUR L'AUDIT
CPU	AMD Ryzen 7 9700X (Zen 5) — 8 cœurs / 16 threads, 3,8 GHz (boost 5,5 GHz)	Boucle de simulation mono-thread : un seul cœur sollicité
Caches	L1d 48 Ko + L1i 32 Ko par cœur • L2 1 Mo par cœur • L3 32 Mo partagé	Les données chaudes (positions : 160 Ko) tiennent en L2
Mémoire	32 Go DDR5-5600 (2 × 16 Go Micron, CL36)	Pas de contrainte de capacité
Système	Windows 11 Professionnel (build 26300), x86_64	Mesures sous Windows, pas sous Linux
Runtime	Go 1.27.0 windows/amd64 (compilateur gc, inlining et escape analysis activés)	Versions identiques pour baseline et final
Outils	benchstat (x/perf) • Hyperfine 1.20.0 • go tool pprof • fieldalignment	

Protocole

MESURE	MÉTHODE	PARAMÈTRES
Coût d'un tick	`go test -bench`, 1 op = 1 tick, état restauré depuis un snapshot tous les 100 ticks pour une charge stable	n = 10 séries • benchstat (Mann-Whitney), seuil p < 0,05
Coût d'une frame réseau	Télémétrie + encodage de la frame complète (10 000 clients, ~360 Ko de JSON)	idem
Bout en bout	Hyperfine sur le binaire en mode headless : 5 000 ticks de simulation, du lancement à la sortie	`--warmup 3 --runs 10 -N` (sans shell intermédiaire)
Profils	pprof CPU et allocations sur les mêmes 5 000 ticks, plus un profil live de 20 s sur le serveur réel	Échantillonnage 100 Hz
Non-régression	Empreinte FNV-1a de tout l'état (positions, alcoolémie, timers, flaques, RNG) après 4 000 ticks	Doit rester `0xaa36b4f0b73c614c`

Deux scénarios de charge

Le premier constat de l'audit concerne le benchmark fourni : il ne mesurait que le début de soirée, quand aucune flaque n'existe. Or les flaques n'apparaissent qu'après ~2 500 ticks et leur coût domine ensuite. Deux scénarios figés ont donc été créés.

SCÉNARIO	ÉTAT	TICK NAÏF
early — début de soirée	Tick 0, aucune flaque	1,645 ms
rush — régime de croisière	Tick 5 000, ~450 flaques actives, 214 clients en train de vomir	5,846 ms

Baseline mesurée avec Hyperfine (5 000 ticks, version naïve)

13,81 s	13,78 s	± 0,095 s	0,0090 s²
Moyenne	Médiane	Écart-type (σ)	Variance (σ²)

Min 13,71 s • max 13,95 s • coefficient de variation 0,69 %. La dispersion est faible : le protocole (machine au repos, warmup) suffit à isoler le bruit.

2. Diagnostic de la baseline

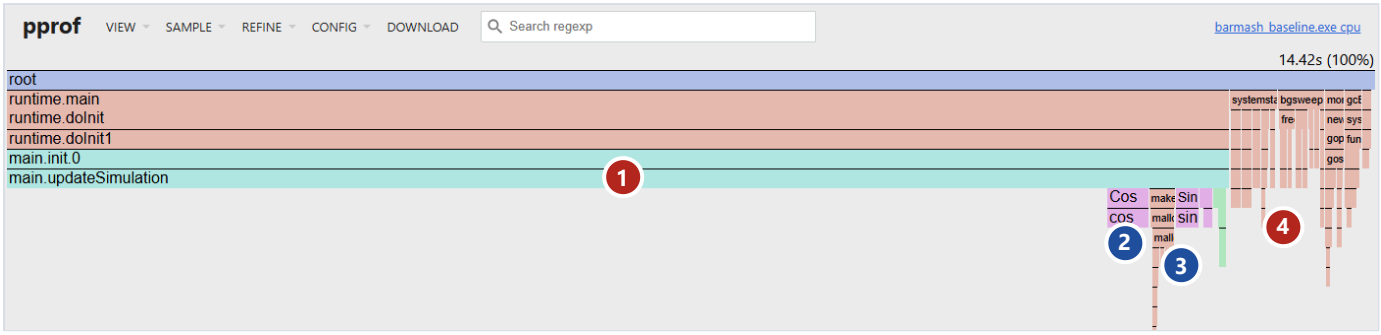


Figure 1 — Flamegraph CPU réel de la version naïve (pprof, 5 000 ticks headless, `perf/campaign/baseline_cpu.pprof`). Total 14,42 s. Repères : (1) plateau de `updateSimulation`, où les boucles des flaques et de la bousculade sont écrites en ligne (72,6 % à elles deux) ; (2) `Cos` / `Sin` ; (3) `makeslice` / `mallocgc` ; (4) tours du GC (`bgsweep`, `gcBgMarkWorker`).

Le flamegraph ne montre qu'un grand plateau, car les boucles chaudes sont écrites en ligne dans `updateSimulation`. Le détail vient donc du profil ligne à ligne :

REPÈRE	ZONE DU HOT PATH	5 000 TICKS	RUSH	CAUSE IDENTIFIÉE
1	**Scan des flaques** — chaque client teste chaque flaque	**39,5 %**	**~60 %**	O(N × P) : 10 000 × ~450 = 4,5 M tests et racines carrées par tick
1	**Scan de bousculade** — 60 voisins par client	**33,1 %**	**~33 %**	600 k lectures de `*Client` de 96 o via pointeurs ; `math.Sqrt` sur tous les voisins
2	Trigonométrie du déplacement	4,7 %	~5 %	`math.Cos` + `math.Sin` séparés pour chaque client mobile
3	Allocations dans la boucle	1,9 %	~2 %	`makeslice` : `make([]BarDistance)`, `neighborsCrowd`
4	Garbage Collector (balayage + marquage)	~5 %	—	`bgsweep` 3,4 % + `gcBgMarkWorker` 1,9 % : conséquence directe de 3
	Recherche du bar le plus proche	2,4 %	—	8 racines carrées par client assoiffé

Part du temps CPU total (14,42 s) attribuée ligne à ligne avec `pprof -top -lines`. Le scan des flaques pèse moins sur 5 000 ticks parce que la première moitié de la soirée n'en contient aucune.

Mémoire et Garbage Collector

INDICATEUR (VERSION NAÏVE)	VALEUR	ORIGINE
Allocations par tick	423 Ko • 3 385 allocs	`make([]BarDistance, len(bars))` par client assoiffé (taille non constante → tas)
Allocations par frame réseau	603 Ko • 7 allocs	`[]ClientPoint` + `json.Marshal` + conversion `string(data)`
Coût d'une frame réseau	1,224 ms	Réflexion de `encoding/json` sur 10 000 structures
5 000 ticks sans serveur	697 cycles GC • 1 820 Mo alloués	Soit ~364 Ko de déchets par tick, collectés en continu (repère 4)
Serveur réel, 25 s	528 cycles GC • 1 611 Mo alloués	S'y ajoute `runtime.ReadMemStats` (pause Stop-The-World) à chaque tick

Lecture

- Le goulot n°1 est **algorithmique** (scan des flaques), pas mémoire : il ne pèse rien en début de soirée et passe à 60 % du temps en croisière.
- Le goulot n°2 combine calcul (racines carrées inutiles) et accès mémoire indirects.
- Côté réseau, la frame coûte presque autant qu'un tick et génère l'essentiel des déchets mémoire.

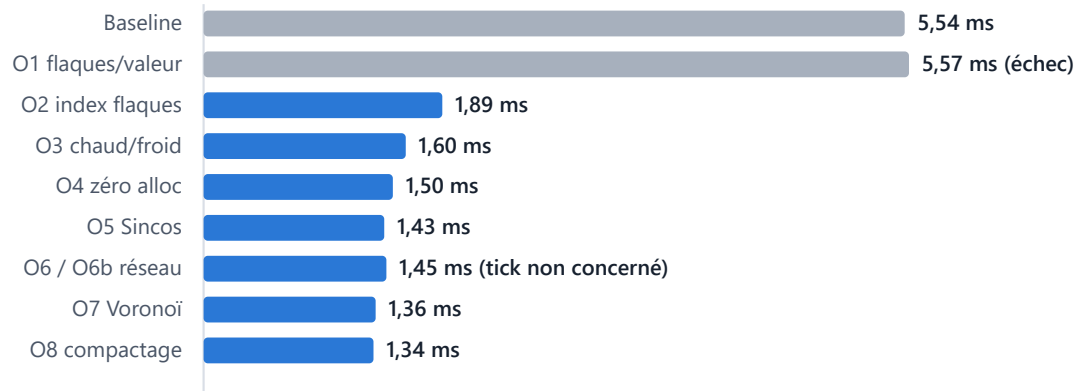
3. Journal d'optimisation

Les actions sont ordonnées selon la loi d'Amdahl : d'abord la zone qui pèse le plus. Chaque ligne correspond à un commit et à une série benchstat (n = 10) comparée à l'étape précédente. Les écarts non significatifs (p ≥ 0,05) sont notés « ~ ».

#	ACTION	HYPOTHÈSE MATÉRIELLE	EARLY	RUSH	VERDICT
Calcul CPU et algorithmique					
O2	Index spatial des flaques (grille 32 × 32, tri par comptage)	4,5 M tests → ~4 candidats par client	~	−66,1 %	retenu
O5	`math.Sincos` au lieu de `Cos` + `Sin`	Une seule réduction d'argument	−8,2 %	−4,6 %	retenu
O7	Table précalculée du bar le plus proche (zones de Voronoï)	8 racines carrées → 1 ; table de 6,4 Ko en L1	−8,9 %	−6,2 %	retenu
Mémoire et localité de cache					
O1	Flaques stockées par valeur, expiration FIFO	Supprimer les sauts de pointeurs	+1,4 %	+0,6 %	échec
O3	Positions isolées dans un tableau contigu de 16 o + rejet rapide par boîte englobante	160 Ko en L2, 4 positions par ligne de 64 o ; 97 % des voisins rejetés sans `Sqrt`	−21,0 %	−15,5 %	retenu
O4	Suppression des allocations par tick	Plus de pression GC dans la boucle	−9,9 %	−6,1 %	0 alloc
O8	Compactage de `Client` : 80 o → 48 o (tri des champs, types réduits)	Clients + positions (640 Ko) dans le L2	~	~	neutre
I/O réseau					
O6	Encodeur JSON écrit à la main dans un buffer réutilisé, `runtime/metrics`, écriture WebSocket directe	0 déchet par frame, plus de pause Stop-The-World		frame −66,5 %	retenu
O6b	Table des entiers 0..2047 précalculée	40 000 entiers écrits en un store de 4 o		frame −60,5 %	retenu

Contrôles systématiques : empreinte bit à bit inchangée à chaque étape ; frame octet-identique à `encoding/json` (le front n'a pas été modifié) ; index, table de Voronoï et `Sincos` comparés à l'implémentation naïve sur plusieurs millions de points.

Évolution du coût d'un tick en croisière (scénario rush)



Valeurs médianes benchstat, mesurées étape par étape pendant le développement. L'essentiel du gain (−66 %) vient d'une seule action, O2 : c'était le plateau le plus large du profil. Le comparatif final de la section 5 est refait dans une même session, d'où de légers écarts avec ces valeurs.

4. Tentatives infructueuses et reproductibilité

Échec n°1 — Rejet par boîte englobante écrit avec `||` : +52 % de temps

VARIANTE (SCAN DE BOUSCULADE)	EARLY	RUSH
Avant (séparation chaud/froid seule)	1,40 ms	—
`if ndx >= 22 || ndx <= -22 { continue }`	2,28 ms (+52 %)	2,72 ms (+44 %)
`if math.Abs(ndx) >= 22 { continue }`	1,14 ms (−19 %)	1,60 ms

- **Hypothèse :** éviter la racine carrée sur les voisins lointains. Mesuré : 97 % des voisins sont éliminés dès le premier test.
- **Cause :** le premier saut dépend du **signe** de dx, qui est aléatoire. Le prédicteur de branchement se trompe une fois sur deux : 600 k voisins × 50 % × ~15 cycles de vidage du pipeline ≈ **0,9 ms**, soit exactement l'écart mesuré (+0,88 ms).
- **Correction :** `math.Abs` est un simple masque de bit, sans branchement. Il ne reste qu'un saut, juste dans 97 % des cas. La même condition logique coûte +60 % ou −19 % selon son écriture.

Échec n°2 — O1 : flaques par valeur (gain nul)

- Hypothèse : les ~450 `*VomitPuddle` dispersés provoquent des défauts de cache.
- Mesure : +1,4 % early, +0,6 % rush ($p < 0{,}01$).
- Cause : les flaques étaient allouées à la suite et tiennent en 18 Ko, donc dans le L1d (48 Ko). Le problème était le nombre de tests, réglé ensuite par O2.

Échec n°3 — O8 : struct de 80 o → 48 o (neutre)

- Hypothèse : tenir clients + positions dans le L2.
- Mesure : early $p = 0{,}063$, rush $p = 0{,}481$ → pas d'effet.
- Cause : les clients sont lus **séquentiellement** ; le prefetcher suit le flux et l'ancien jeu (960 Ko) tenait déjà en L2. Conservé pour l'empreinte mémoire (−40 %).

Biais de mesure repéré : alignement du code

Après O6, le tick early a pris +2,8 % alors que la boucle n'avait pas changé. Un A/B entrelacé des deux binaires confirme l'écart ($p = 0{,}000$), et la comparaison du code machine montre que seuls le padding (`NOP`) et un pas d'adresse ont bougé : la boucle chaude a été réalignée. C'est un effet de placement du code, pas un effet algorithmique. D'où la règle suivie ensuite : comparer en A/B entrelacé dès que l'écart attendu est sous ~5 %.

Reproduire la campagne en une commande

```
# Prérequis : Go 1.27, benchstat, hyperfine. Durée ~10 min.
./run_benchmarks.sh
```

ÉTAPE	CE QUE FAIT LE SCRIPT	SORTIE (PERF/CAMPAIGN/)
1–2	Reconstruit la baseline depuis git (`343a4ee`) dans un worktree et compile les deux binaires avec le même harnais headless	`environment.txt`
3	Tests : empreinte bit à bit et oracles d'équivalence	`tests.txt`
4	Benchmarks Go baseline et final dans la même session, puis benchstat	`bench_*.txt` , `benchstat.txt`
5	Hyperfine sur 5 000 ticks + vérification que l'état final est identique	`hyperfine.json/.md` , `state_*.txt`
6	Profils pprof CPU et allocations des deux versions	`*_cpu.pprof` , `*_allocs.pprof`

Historique complet : un commit par action, avec ses séries brutes dans `perf/bench/` . Les règles imposées aux assistants IA utilisés pendant le projet sont dans `constitution.md` à la racine du dépôt.

5. Synthèse et conclusion

×4,5	×9,9	0 alloc	528 → 4
Tick en croisière plus rapide	Frame réseau plus rapide	par tick et par frame	cycles GC en 25 s de service

Baseline vs version finale

INDICATEUR	BASELINE	FINALE	ÉCART	SOURCE
Tick, début de soirée (early)	1,645 ms	0,912 ms	−44,5 %	benchstat n=10, p=0,000
Tick, croisière (rush)	5,846 ms	1,297 ms	−77,8 % (×4,5)	benchstat n=10, p=0,000
Frame réseau (télémétrie + encodage)	1,224 ms	0,123 ms	−89,9 % (×9,9)	benchstat n=10, p=0,000
Allocations par tick	423 Ko • 3 385	0 • 0	−100 %	benchstat
Allocations par frame	603 Ko • 7	0 • 0	−100 %	benchstat
5 000 ticks de bout en bout (moyenne ± σ)	13,81 ± 0,09 s	5,55 ± 0,02 s	×2,49	Hyperfine, 3 warmup + 10 runs
GC sur 5 000 ticks • mémoire allouée cumulée	697 cycles • 1 820 Mo	0 cycle • 0,94 Mo	−100 %	runtime/metrics (headless)
Serveur réel, 25 s : CPU consommé • cycles GC	2 610 ms • 528	840 ms • 4	−68 %	pprof live + client WebSocket
Taille d'un client en mémoire	96 o (via pointeur)	48 o + 16 o (position)	−33 %	`unsafe.Sizeof`
État de la simulation après 5 000 ticks	**6 052 sobres • 214 vomisseurs • 438 flaques • 141 838 bières**		**identique**	**empreinte bit à bit**

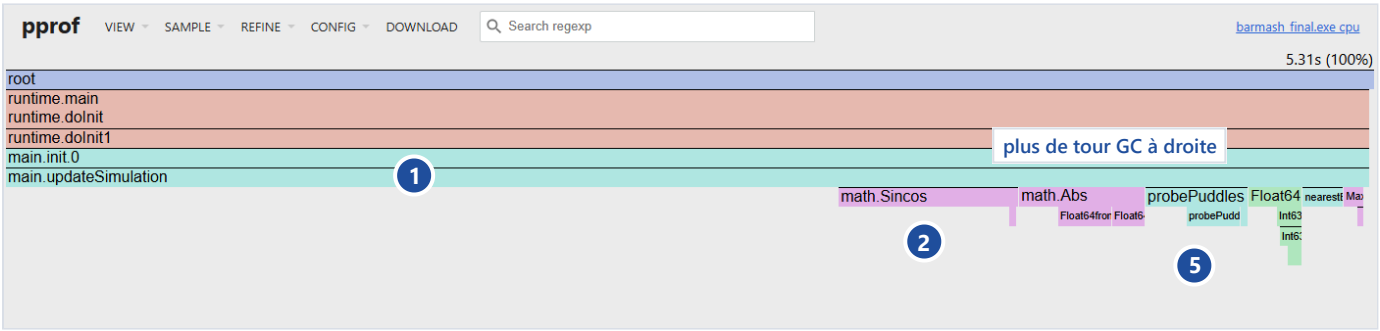


Figure 2 — Flamegraph CPU de la version finale, sur les mêmes 5 000 ticks (total 5,31 s contre 14,42 s). Repères : (1) bousculade, qui reste le premier poste ; (2) `math.Sincos` (12,4 %) ; (5) `probePuddles`, le scan des flaques via l'index (7,5 % contre 39,5 %). Plus aucune tour du GC.

Conclusion

Action la plus rentable : O2, l'index des flaques. À elle seule, elle divise le tick de croisière par 2,95. C'est aussi celle que le profil désignait le plus clairement. Côté réseau, l'encodeur manuel (O6 + O6b) divise le coût d'une frame par près de 10.

Accent mis : d'abord le **CPU** (algorithmique, puis branchements et racines carrées), puis la **mémoire / GC** (zéro allocation, flaques et clients sans pointeurs, −40 % sur la struct client). Le serveur tient ses 30 ticks/s avec une marge énorme : 1,3 ms utilisées sur 33,3 ms de budget.

Ce qui n'a pas marché : l'erreur la plus instructive est le rejet par boîte englobante écrit avec `||`. Une optimisation « évidente » sur le papier a ralenti le tick de 52 % à cause du prédicteur de branchement. Les deux optimisations purement mémoire (O1, O8) n'ont rien apporté au CPU : dans ce projet, les données tenaient déjà en cache.

Suite : la concurrence n'a pas été traitée. Paralléliser la boucle impose de séparer une phase de lecture (forces, flaques) d'une phase d'écriture pour garder le déterminisme du RNG. Un format binaire pour la frame (~360 Ko de JSON aujourd'hui) réduirait aussi la bande passante réseau.